POLYBOT

System Design Document

Architecture, Data Flow & Component Specifications

Version	v1.0
Date	March 2025
Status	Ready for Claude Code
Author	Polybot Team

© 2025 Polybot. All rights reserved.

1. System Overview

Polybot is a Node.js monorepo designed around three layers: the core infrastructure layer (wallet, risk, state, logging), the agent layer (6 specialized Claude-powered market scanners), and the interface layer (REST API + WebSocket dashboard). All components communicate through a central in-memory EventEmitter-based StateStore.

2. Architecture

2.1 Component Map

Component	File	Responsibility
Entry Point	index.js	Boot sequence, agent orchestration, cron jobs
BaseAgent	agents/baseAgent.js	Claude analysis, order execution, scan loop
Specialized Agents	agents/specializedAgents.js	Market category filtering per agent type
PolymarketClient	core/polymarketClient.js	CLOB API, wallet, orderbook, positions
RiskManager	core/riskManager.js	Trade approval, Kelly sizing, halt logic
NewsFetcher	core/newsFetcher.js	Real-time news/data from 3 external APIs
StateStore	core/stateStore.js	Shared in-memory state + event bus
Logger	core/logger.js	Winston: console + rotating file logs
Dashboard Server	dashboard/server.js	Express HTTP + WebSocket real-time push
Dashboard UI	dashboard/public/index.html	Single-page live trading dashboard

2.2 Data Flow — Single Scan Cycle

The following describes the complete data flow for one agent scan cycle:

1. Agent timer fires (interval reached)
2. Agent calls stateStore.engineRunning — aborts if false
3. Agent calls riskManager.engineHalted — aborts if true
4. Agent calls getMarkets() → PolymarketClient.getActiveMarkets() → Gamma API
5. Agent calls newsFetcher.getContextForCategory() → external news APIs (cached 60s)
6. For each target market (max 3): Agent builds Claude prompt with market + news context
7. Agent calls Anthropic API (claude-sonnet-4) → receives JSON decision
8. Agent calls riskManager.approve(decision) → returns {approved, stake, reason}
9. If approved: Agent calls polymarketClient.placeOrder() → CLOB API → Polygon chain
10. Trade recorded in stateStore → WebSocket broadcast → Dashboard updates

© 2025 Polybot. All rights reserved.

2.3 State Management

StateStore is a singleton EventEmitter. It holds all runtime state and emits typed events that the WebSocket server listens to for broadcasting. No database is used in v1 — state is in-memory and resets on process restart. Trade log is preserved in logs/trades.log.

Key state events emitted:

- trade — new trade opened
- trade_update — existing trade status changed
- pnl — P&L value updated
- agent — agent status/metadata updated
- news — new news/signal item
- state — general state change (engine start/stop, wallet update)

3. External Integrations

3.1 Polymarket CLOB API

Endpoint	Purpose	Notes
GET /markets (Gamma)	Fetch active markets	Filtered by active=true, sorted by volume
GET /book	Get orderbook	Requires token_id
GET /positions	Current wallet positions	Requires wallet auth headers
POST /order	Place order	FOK type, signed with wallet private key
DELETE /order/:id	Cancel order	Requires orderId

3.2 Anthropic Claude API

Model: claude-sonnet-4-20250514. Each market analysis call uses max_tokens: 300. System prompt is shared across all agents (defined in BaseAgent). The prompt enforces JSON-only output — no markdown wrapping. Responses are parsed with JSON.parse() after stripping code fences.

Claude decision schema:

- trade — "YES" | "NO" | "SKIP"
- true_probability — number 0–100
- confidence — number 0–100
- edge — number (true_prob - market_prob)
- risk_level — "LOW" | "MEDIUM" | "HIGH"
- time_sensitivity — "LOW" | "MEDIUM" | "HIGH"
- reason — max 12 words

© 2025 Polybot. All rights reserved.

- key_factor — single strongest signal
- warning — biggest risk to this trade

3.3 News APIs

Source	Category	Auth Required
CryptoCompare /news	Crypto	No — free tier, no key needed
CoinGecko /simple/price	Crypto	No — free tier, no key needed
Open-Meteo /forecast	Weather	No — fully free, no key needed
GNews /search	General/Econ/Politics	Optional GNEWS_API_KEY (100 req/day free)

3.4 Response Caching

NewsFetcher implements a 60-second in-memory TTL cache per endpoint. This prevents duplicate API calls when multiple agents scan simultaneously. Cache is keyed by endpoint + parameters and stores {data, ts}.

4. Risk Engine Design

4.1 Kelly Stake Sizing

Stake is calculated using a fractional Kelly formula (safety factor 0.25x):

kellyFraction = (|edge| / 100) × (confidence / 100) × 0.25

rawStake = basePot × kellyFraction

basePot is proxied as MAX_STAKE_USD × 10. rawStake is then adjusted by risk level modifier (HIGH: 0.4x, MEDIUM: 0.7x, LOW: 1.0x) and clamped to [MIN_STAKE_USD, MAX_STAKE_USD].

4.2 Approval Gate

RiskManager.approve() evaluates conditions in this order — first failure returns immediately:

11. engineHalted check
12. dailyPnl <= -dailyLossLimit check
13. openTradeCount >= maxOpenTrades check
14. decision.trade === SKIP check
15. |edge| < minEdge check
16. confidence < minConfidence check
17. risk_level === HIGH and |edge| < 10 check

Only if all pass: calcStake() is called and {approved: true, stake} is returned.

© 2025 Polybot. All rights reserved.

5. Dashboard API

5.1 REST Endpoints

Method	Path	Description
GET	/api/state	Full state snapshot — used on dashboard load
POST	/api/engine/stop	Sets stateStore.engineRunning = false
POST	/api/engine/start	Sets engineRunning = true, resumes riskManager if needed
POST	/api/trade/manual	Emits manual_trade event — agent picks it up and analyzes
POST	/api/risk/reset-daily	Resets dailyPnl to 0, resumes engine if daily-limit halted

5.2 WebSocket Message Types

Type	Payload
snapshot	Full state object — sent once on connection
state	Full state object — sent on any general state change
trade	Single trade object — sent when trade opens
trade_update	Partial trade patch — sent when trade status changes
pnl	{value, history[]} — sent on every P&L change
agent	{name, data} — sent on agent status update
news	{agent, text, type, ts} — news or signal item

6. Directory Structure

Path	Contents
index.js	Main entry — boot, agents, cron, graceful shutdown
agents/baseAgent.js	BaseAgent class — Claude analysis + execution
agents/specializedAgents.js	6 agent subclasses with category-specific market filters
core/polymarketClient.js	Polymarket CLOB + Gamma API client
core/riskManager.js	Kelly sizing, approval gate, halt/resume, stats
core/newsFetcher.js	Multi-source news/data fetcher with TTL cache
core/stateStore.js	EventEmitter singleton — shared live state
core/logger.js	Winston logger — console + rotating file

© 2025 Polybot. All rights reserved.

dashboard/server.js	Express REST + WebSocket server
dashboard/public/index.html	Single-file dashboard SPA
.env.example	Environment variable template
logs/	trades.log, error.log (auto-created, auto-rotated)

7. Dependencies

Package	Version	Purpose
@anthropic-ai/sdk	^0.20.0	Claude AI API client
ethers	^6.11.0	Polygon wallet, USDC balance, order signing
axios	^1.6.0	HTTP client for all external API calls
express	^4.18.0	REST API server for dashboard
ws	^8.16.0	WebSocket server for real-time dashboard push
node-cron	^3.0.3	Midnight daily P&L reset
winston	^3.11.0	Structured logging with file rotation
dotenv	^16.4.0	Environment variable loading

© 2025 Polybot. All rights reserved.